

```
1 #include <windows.h>
2 #include <stdio.h> // for FILE I/O
3
4 #include <gl\glew.h> // for GLSL extensions IMPORTANT : This Line Should Be Before #include<gl\gl.h> And #include<gl\glu.h>
5
6 #include <gl/GL.h>
7
8 #include "vmath.h"
9
10 #pragma comment(lib, "glew32.lib")
11 #pragma comment(lib, "opengl32.lib")
12
13 #define WIN_WIDTH 800
14 #define WIN_HEIGHT 600
15
16 using namespace vmath;
17
18 enum
19 {
20     VDG_ATTRIBUTE_VERTEX = 0,
21     VDG_ATTRIBUTE_COLOR,
22     VDG_ATTRIBUTE_NORMAL,
23     VDG_ATTRIBUTE_TEXTURE0,
24 };
25
26 //Prototype Of WndProc() declared Globally
27 LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
28
29 //Global variable declarations
30 FILE *gpFile = NULL;
31
32 HWND ghwnd = NULL;
33 HDC ghdc = NULL;
34 HGLRC ghrc = NULL;
35
36 DWORD dwStyle;
37 WINDOWPLACEMENT wpPrev = { sizeof(WINDOWPLACEMENT) };
38
39 bool gbActiveWindow = false;
40 bool gbEscapeKeyIsPressed = false;
41 bool gbFullscreen = false;
42
43 GLuint gVertexShaderObject;
44 GLuint gFragmentShaderObject;
45 GLuint gShaderProgramObject;
46
47 GLuint gVao;
48 GLuint gVbo;
49 GLuint gMVPUniform;
50
51 mat4 gOrthographicProjectionMatrix;
52
53 //main()
54 int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int iCmdShow)
```

```
55 {
56     //function prototype
57     void initialize(void);
58     void uninitialize(void);
59     void display(void);
60
61     //variable declaration
62     WNDCLASSEX wndclass;
63     HWND hwnd;
64     MSG msg;
65     TCHAR szClassName[] = TEXT("OpenGLPP");
66     bool bDone = false;
67
68     //code
69     // create log file
70     if (fopen_s(&gpFile, "Log.txt", "w") != 0)
71     {
72         MessageBox(NULL, TEXT("Log File Can Not Be Created\nExiting ..."),
73             TEXT("Error"), MB_OK | MB_TOPMOST | MB_ICONSTOP);
74         exit(0);
75     }
76     else
77     {
78         fprintf(gpFile, "Log File Is Successfully Opened.\n");
79     }
80
81     //initializing members of struct WNDCLASSEX
82     wndclass.cbSize = sizeof(WNDCLASSEX);
83     wndclass.style = CS_HREDRAW | CS_VREDRAW | CS_OWNDC;
84     wndclass.cbClsExtra = 0;
85     wndclass.cbWndExtra = 0;
86     wndclass.hInstance = hInstance;
87     wndclass.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
88     wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
89     wndclass.hCursor = LoadCursor(NULL, IDC_ARROW);
90     wndclass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
91     wndclass.lpfnWndProc = WndProc;
92     wndclass.lpszClassName = szClassName;
93     wndclass.lpszMenuName = NULL;
94     //Registering Class
95     RegisterClassEx(&wndclass);
96
97     //Create Window
98     //Parallel to glutInitWindowSize(), glutInitWindowPosition() and
99     //glutCreateWindow() all three together
100     hwnd = CreateWindow(szClassName,
101         TEXT("OpenGL Programmable Pipeline Window"),
102         WS_OVERLAPPEDWINDOW,
103         100,
104         100,
105         WIN_WIDTH,
106         WIN_HEIGHT,
107         NULL,
108         NULL,
109         hInstance,
110         NULL);
```

```

109
110     ghwnd = hwnd;
111
112     ShowWindow(hwnd, iCmdShow);
113     SetForegroundWindow(hwnd);
114     SetFocus(hwnd);
115
116     //initialize
117     initialize();
118
119     //Message Loop
120     while (bDone == false) //Parallel to glutMainLoop();
121     {
122         if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
123         {
124             if (msg.message == WM_QUIT)
125                 bDone = true;
126             else
127             {
128                 TranslateMessage(&msg);
129                 DispatchMessage(&msg);
130             }
131         }
132         else
133         {
134             // rendering function
135             display();
136
137             if (gbActiveWindow == true)
138             {
139                 if (gbEscapeKeyIsPressed == true) //Continuation to
140                     glutLeaveMainLoop();
141                 bDone = true;
142             }
143         }
144     }
145     uninitialize();
146
147     return((int)msg.wParam);
148 }
149
150 //WndProc()
151 LRESULT CALLBACK WndProc(HWND hwnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
152 {
153     //function prototype
154     void resize(int, int);
155     void ToggleFullscreen(void);
156     void uninitialize(void);
157
158     //variable declarations
159     static WORD xMouse = NULL;
160     static WORD yMouse = NULL;
161
162     //code
163     switch (iMsg)

```

```

164     {
165         case WM_ACTIVATE:
166             if (HIWORD(wParam) == 0) //if 0, the window is active
167                 gbActiveWindow = true;
168             else //if non-zero, the window is not active
169                 gbActiveWindow = false;
170             break;
171         case WM_ERASEBKGND:
172             return(0);
173         case WM_SIZE: //Parallel to glutReshapeFunc();
174             resize(LOWORD(lParam), HIWORD(lParam)); //Parallel to glutReshapeFunc ↗
175             (resize);
176             break;
177         case WM_KEYDOWN: //Parallel to glutKeyboardFunc();
178             switch (wParam)
179             {
180                 case VK_ESCAPE: //case 27
181                     if (gbEscapeKeyPressed == false)
182                         gbEscapeKeyPressed = true; //Parallel to glutLeaveMainLoop ↗
183                     ();
184                     break;
185                 case 0x46: //for 'f' or 'F'
186                     if (gbFullscreen == false)
187                     {
188                         ToggleFullscreen();
189                         gbFullscreen = true;
190                     }
191                     else
192                     {
193                         ToggleFullscreen();
194                         gbFullscreen = false;
195                     }
196                     break;
197                 default:
198                     break;
199             }
200         case WM_LBUTTONDOWN: //Parallel to glutMouseFunc();
201             break;
202         case WM_CLOSE: //Parallel to glutCloseFunc();
203             uninitialize(); //Parallel to glutCloseFunc(uninitialize);
204             break;
205         case WM_DESTROY:
206             PostQuitMessage(0);
207             break;
208         default:
209             break;
210     }
211     return(DefWindowProc(hwnd, iMsg, wParam, lParam));
212 }
213 void ToggleFullscreen(void)
214 {
215     //variable declarations
216     MONITORINFO mi;
217

```

```

218 //code
219 if (gbFullscreen == false)
220 {
221     dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
222     if (dwStyle & WS_OVERLAPPEDWINDOW)
223     {
224         mi = { sizeof(MONITORINFO) };
225         if (GetWindowPlacement(ghwnd, &wpPrev) && GetMonitorInfo
226             (MonitorFromWindow(ghwnd, MONITORINFOF_PRIMARY), &mi))
227         {
228             SetWindowLong(ghwnd, GWL_STYLE, dwStyle &
229                 ~WS_OVERLAPPEDWINDOW);
230             SetWindowPos(ghwnd, HWND_TOP, mi.rcMonitor.left,
231                 mi.rcMonitor.top, mi.rcMonitor.right - mi.rcMonitor.left,
232                 mi.rcMonitor.bottom - mi.rcMonitor.top, SWP_NOZORDER |
233                 SWP_FRAMECHANGED);
234         }
235     }
236     ShowCursor(FALSE);
237 }
238 else
239 {
240     //code
241     SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
242     SetWindowPlacement(ghwnd, &wpPrev);
243     SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
244         SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
245     ShowCursor(TRUE);
246 }
247 }
248
249 //FUNCTION DEFINITIONS
250 void initialize(void)
251 {
252     //function prototypes
253     void uninitialize(void);
254     void resize(int, int);
255
256     //variable declarations
257     PIXELFORMATDESCRIPTOR pfd;
258     int iPixelFormatIndex;
259
260     //code
261     ZeroMemory(&pfd, sizeof(PIXELFORMATDESCRIPTOR));
262
263     //Initialization of structure 'PIXELFORMATDESCRIPTOR'
264     //Parallel to glutInitDisplayMode()
265     pfd.nSize = sizeof(PIXELFORMATDESCRIPTOR);
266     pfd.nVersion = 1;
267     pfd.dwFlags = PFD_DRAW_TO_WINDOW | PFD_SUPPORT_OPENGL | PFD_DOUBLEBUFFER;
268     pfd.iPixelFormat = PFD_TYPE_RGBA;
269     pfd.cColorBits = 32;
270     pfd.cRedBits = 8;
271     pfd.cGreenBits = 8;

```

```
268     pfd.cBlueBits = 8;
269     pfd.cAlphaBits = 8;
270     pfd.cDepthBits = 32;
271
272     ghdc = GetDC(ghwnd);
273
274     //choose a pixel format which best matches with that of 'pfd'
275     iPixelFormatIndex = ChoosePixelFormat(ghdc, &pfd);
276     if (iPixelFormatIndex == 0)
277     {
278         ReleaseDC(ghwnd, ghdc);
279         ghdc = NULL;
280     }
281
282     //set the pixel format chosen above
283     if (SetPixelFormat(ghdc, iPixelFormatIndex, &pfd) == false)
284     {
285         ReleaseDC(ghwnd, ghdc);
286         ghdc = NULL;
287     }
288
289     //create OpenGL rendering context
290     ghrc = wglCreateContext(ghdc);
291     if (ghrc == NULL)
292     {
293         ReleaseDC(ghwnd, ghdc);
294         ghdc = NULL;
295     }
296
297     //make the rendering context created above as current n the current hdc
298     if (wglMakeCurrent(ghdc, ghrc) == false)
299     {
300         wglDeleteContext(ghrc);
301         ghrc = NULL;
302         ReleaseDC(ghwnd, ghdc);
303         ghdc = NULL;
304     }
305
306     // GLEW Initialization Code For GLSL ( IMPORTANT : It Must Be Here. Means ?
307     // After Creating OpenGL Context But Before Using Any OpenGL Function )
308     GLenum glew_error = glewInit();
309     if (glew_error != GLEW_OK)
310     {
311         wglDeleteContext(ghrc);
312         ghrc = NULL;
313         ReleaseDC(ghwnd, ghdc);
314         ghdc = NULL;
315     }
316
317     // *** VERTEX SHADER ***
318     // create shader
319     glVertexShaderObject = glCreateShader(GL_VERTEX_SHADER);
320
321     // provide source code to shader
322     const GLchar *vertexShaderSourceCode =
323         "#version 130" \
```

```
323     "\n" \
324     "in vec4 vPosition;" \
325     "uniform mat4 u_mvp_matrix;" \
326     "void main(void)" \
327     "{" \
328     "    gl_Position = u_mvp_matrix * vPosition;" \
329     "}";
330 glShaderSource(gVertexShaderObject, 1, (const GLchar **)
    &vertexShaderSourceCode, NULL);

331
332 // compile shader
333 glCompileShader(gVertexShaderObject);
334 GLint iInfoLogLength = 0;
335 GLint iShaderCompiledStatus = 0;
336 char *szInfoLog = NULL;
337 glGetShaderiv(gVertexShaderObject, GL_COMPILE_STATUS,
    &iShaderCompiledStatus);
338 if (iShaderCompiledStatus == GL_FALSE)
339 {
340     glGetShaderiv(gVertexShaderObject, GL_INFO_LOG_LENGTH,
        &iInfoLogLength);
341     if (iInfoLogLength > 0)
342     {
343         szInfoLog = (char *)malloc(iInfoLogLength);
344         if (szInfoLog != NULL)
345         {
346             GLsizei written;
347             glGetShaderInfoLog(gVertexShaderObject, iInfoLogLength,
                &written, szInfoLog);
348             fprintf(gpFile, "Vertex Shader Compilation Log : %s\n",
                szInfoLog);
349             free(szInfoLog);
350             uninitialized();
351             exit(0);
352         }
353     }
354 }
355
356 // *** FRAGMENT SHADER ***
357 // create shader
358 gFragmentShaderObject = glCreateShader(GL_FRAGMENT_SHADER);
359
360 // provide source code to shader
361 const GLchar *fragmentShaderSourceCode =
362     "#version 130" \
363     "\n" \
364     "out vec4 FragColor;" \
365     "void main(void)" \
366     "{" \
367     "    FragColor = vec4(1.0, 1.0, 1.0, 1.0);" \
368     "}";
369 glShaderSource(gFragmentShaderObject, 1, (const GLchar **)
    &fragmentShaderSourceCode, NULL);

370
371 // compile shader
372 glCompileShader(gFragmentShaderObject);
```

```

373     glGetShaderiv(gFragmentShaderObject, GL_COMPILE_STATUS,
374                  &iShaderCompiledStatus);
375     if (iShaderCompiledStatus == GL_FALSE)
376     {
377         glGetShaderiv(gFragmentShaderObject, GL_INFO_LOG_LENGTH,
378                      &iInfoLogLength);
379         if (iInfoLogLength > 0)
380         {
381             szInfoLog = (char *)malloc(iInfoLogLength);
382             if (szInfoLog != NULL)
383             {
384                 GLsizei written;
385                 glGetShaderInfoLog(gFragmentShaderObject, iInfoLogLength,
386                                  &written, szInfoLog);
387                 fprintf(gpFile, "Fragment Shader Compilation Log : %s\n",
388                      szInfoLog);
389                 free(szInfoLog);
390                 uninitialized();
391                 exit(0);
392             }
393         }
394     }
395
396     // *** SHADER PROGRAM ***
397     // create
398     gShaderProgramObject = glCreateProgram();
399
400     // attach vertex shader to shader program
401     glAttachShader(gShaderProgramObject, gVertexShaderObject);
402
403     // attach fragment shader to shader program
404     glAttachShader(gShaderProgramObject, gFragmentShaderObject);
405
406     // pre-link binding of shader program object with vertex shader position
407     attribute
408     glBindAttribLocation(gShaderProgramObject, VDG_ATTRIBUTE_VERTEX,
409                          "vPosition");
410
411     // link shader
412     glLinkProgram(gShaderProgramObject);
413     GLint iShaderProgramLinkStatus = 0;
414     glGetProgramiv(gShaderProgramObject, GL_LINK_STATUS,
415                   &iShaderProgramLinkStatus);
416     if (iShaderProgramLinkStatus == GL_FALSE)
417     {
418         glGetProgramiv(gShaderProgramObject, GL_INFO_LOG_LENGTH,
419                      &iInfoLogLength);
420         if (iInfoLogLength > 0)
421         {
422             szInfoLog = (char *)malloc(iInfoLogLength);
423             if (szInfoLog != NULL)
424             {
425                 GLsizei written;
426                 glGetProgramInfoLog(gShaderProgramObject, iInfoLogLength,
427                                    &written, szInfoLog);
428                 fprintf(gpFile, "Shader Program Link Log : %s\n", szInfoLog);

```

```
420         free(szInfoLog);
421         uninitialize();
422         exit(0);
423     }
424 }
425 }
426
427 // get MVP uniform location
428 gMVPUniform = glGetUniformLocation(gShaderProgramObject, "u_mvp_matrix");
429
430 // *** vertices, colors, shader attribs, vbo, vao initializations ***
431 const GLfloat triangleVertices[] =
432 { 0.0f, 50.0f, 0.0f, // apex
433   -50.0f, -50.0f, 0.0f, // left-bottom
434    50.0f, -50.0f, 0.0f // right-bottom
435 };
436
437 glGenVertexArrays(1, &gVao);
438 glBindVertexArray(gVao);
439
440 glGenBuffers(1, &gVbo);
441 glBindBuffer(GL_ARRAY_BUFFER, gVbo);
442 glBufferData(GL_ARRAY_BUFFER, sizeof(triangleVertices), triangleVertices,
443             GL_STATIC_DRAW);
444
445 glVertexAttribPointer(VDG_ATTRIBUTE_VERTEX, 3, GL_FLOAT, GL_FALSE, 0,
446                       NULL);
447
448 glEnableVertexAttribArray(VDG_ATTRIBUTE_VERTEX);
449
450 glBindBuffer(GL_ARRAY_BUFFER, 0);
451 glBindVertexArray(0);
452
453 glShadeModel(GL_SMOOTH);
454 // set-up depth buffer
455 glClearDepth(1.0f);
456 // enable depth testing
457 glEnable(GL_DEPTH_TEST);
458 // depth test to do
459 glDepthFunc(GL_LEQUAL);
460 // set really nice perspective calculations ?
461 glHint(GL_PERSPECTIVE_CORRECTION_HINT, GL_NICEST);
462 // We will always cull back faces for better performance
463 glEnable(GL_CULL_FACE);
464
465 // set background color to which it will display even if it will empty.
466     THIS LINE CAN BE IN drawRect().
467 glClearColor(0.0f, 0.0f, 1.0f, 0.0f); // blue
468
469 // set orthographicMatrix to identity matrix
470 gOrthographicProjectionMatrix = mat4::identity();
471
472 // resize
473 resize(WIN_WIDTH, WIN_HEIGHT);
474 }
```

```
473 void display(void)
474 {
475     //code
476     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
477
478     // start using OpenGL program object
479     glUseProgram(gShaderProgramObject);
480
481     // OpenGL Drawing
482     // set modelview & modelviewprojection matrices to identity
483     mat4 modelViewMatrix = mat4::identity();
484     mat4 modelViewProjectionMatrix = mat4::identity();
485
486     // multiply the modelview and orthographic matrix to get modelviewprojection matrix
487     modelViewProjectionMatrix = gOrthographicProjectionMatrix * modelViewMatrix; // ORDER IS IMPORTANT
488
489     // pass above modelviewprojection matrix to the vertex shader in 'u_mvp_matrix' shader variable
490     // whose position value we already calculated in initWithFrame() by using glGetUniformLocation()
491     glUniformMatrix4fv(gMVPUniform, 1, GL_FALSE, modelViewProjectionMatrix);
492
493     // *** bind vao ***
494     glBindVertexArray(gVao);
495
496     // *** draw, either by glDrawTriangles() or glDrawArrays() or glDrawElements()
497     glDrawArrays(GL_TRIANGLES, 0, 3); // 3 (each with its x,y,z ) vertices in triangleVertices array
498
499     // *** unbind vao ***
500     glBindVertexArray(0);
501
502     // stop using OpenGL program object
503     glUseProgram(0);
504
505     SwapBuffers(ghdc);
506 }
507
508 void resize(int width, int height)
509 {
510     //code
511     if (height == 0)
512         height = 1;
513     glViewport(0, 0, (GLsizei)width, (GLsizei)height);
514
515     //glOrtho(left,right,bottom,top,near,far);
516     if (width <= height)
517         gOrthographicProjectionMatrix = ortho(-100.0f, 100.0f, (-100.0f * (height / width)), (100.0f * (height / width)), -100.0f, 100.0f); // co-ordinates written for glVertex3f() are relative to viewing volume of (-100.0f,100.0f,(-100.0f * (height/width)),(100.0f * (height/width)),-100.0f,100.0f)
518     else
```

```
519     gOrthographicProjectionMatrix = ortho(-100.0f, 100.0f, (-100.0f *  
        (width / height)), (100.0f * (width / height)), -100.0f, 100.0f); //  
        co-ordinates written for glVertex3f() are relative to viewing volume  
        of (-100.0f,100.0f,(-100.0f * (height/width)),(100.0f * (height/  
        width)), -100.0f,100.0f)  
520 }  
521  
522 void uninitialize(void)  
523 {  
524     //UNINITIALIZATION CODE  
525     if (gbFullscreen == true)  
526     {  
527         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);  
528         SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);  
529         SetWindowPlacement(ghwnd, &wpPrev);  
530         SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |  
            SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);  
531  
532         ShowCursor(TRUE);  
533     }  
534  
535     // destroy vao  
536     if (gVao)  
537     {  
538         glDeleteVertexArrays(1, &gVao);  
539         gVao = 0;  
540     }  
541  
542     // destroy vbo  
543     if (gVbo)  
544     {  
545         glDeleteBuffers(1, &gVbo);  
546         gVbo = 0;  
547     }  
548  
549     // detach vertex shader from shader program object  
550     glDetachShader(gShaderProgramObject, gVertexShaderObject);  
551     // detach fragment shader from shader program object  
552     glDetachShader(gShaderProgramObject, gFragmentShaderObject);  
553  
554     // delete vertex shader object  
555     glDeleteShader(gVertexShaderObject);  
556     gVertexShaderObject = 0;  
557     // delete fragment shader object  
558     glDeleteShader(gFragmentShaderObject);  
559     gFragmentShaderObject = 0;  
560  
561     // delete shader program object  
562     glDeleteProgram(gShaderProgramObject);  
563     gShaderProgramObject = 0;  
564  
565     // unlink shader program  
566     glUseProgram(0);  
567  
568     //Deselect the rendering context  
569
```

```
570     wglMakeCurrent(NULL, NULL);
571
572     //Delete the rendering context
573     wglDeleteContext(ghrc);
574     ghrc = NULL;
575
576     //Delete the device context
577     ReleaseDC(ghwnd, ghdc);
578     ghdc = NULL;
579
580     if (gpFile)
581     {
582         fprintf(gpFile, "Log File Is Successfully Closed.\n");
583         fclose(gpFile);
584         gpFile = NULL;
585     }
586 }
587
```